

Section 1: Edge Impulse Results

Link: <https://studio.edgeimpulse.com/public/995541/live>

1. Raw inputs: State the raw inputs used in the Edge Impulse project. **The raw inputs are time-series X, Y, and Z acceleration values.**
2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier. **It takes in features from the X, Y, and Z acceleration. There are 33 input features to the classifier. 5 actual features used are: accX RMS, accX Peak 1 Height, accY Peak 3 Height, accX Peak 1 Frequency, and accY RMS.**
3. NN classifier outputs: State the outputs of the NN classifier. **Nominal and off.**
4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector. **There are 4 features used: accX RMS, accX Peak 1 Height, accY spectral power 2-5HZ, accZ spectral power 2-5Hz.**
5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

```

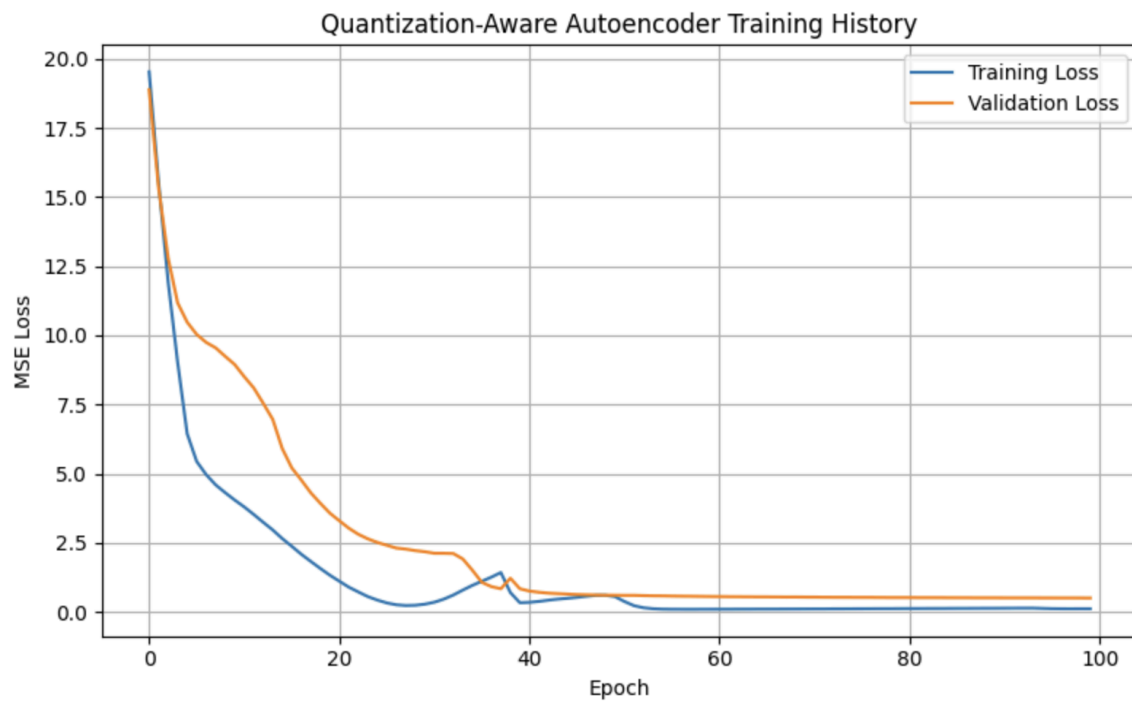
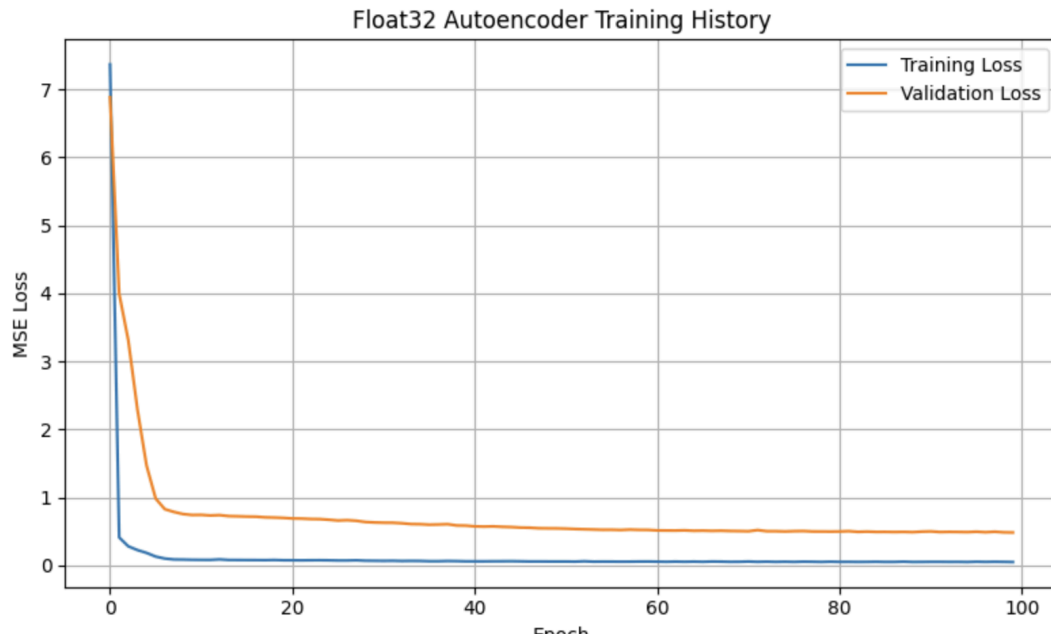
off: 0.195312
Anomaly prediction: 194.992691
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 39 ms, inference 6 ms, anomaly 548 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.289063
  off: 0.710937
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 582 us, anomaly 544 us, postprocessing 67 us
#Classification predictions:
  nominal: 0.656250
  off: 0.343750
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 580 us, anomaly 525 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.375000
  off: 0.625000
Anomaly prediction: 15.819454
Starting inferencing in 2 seconds...

```

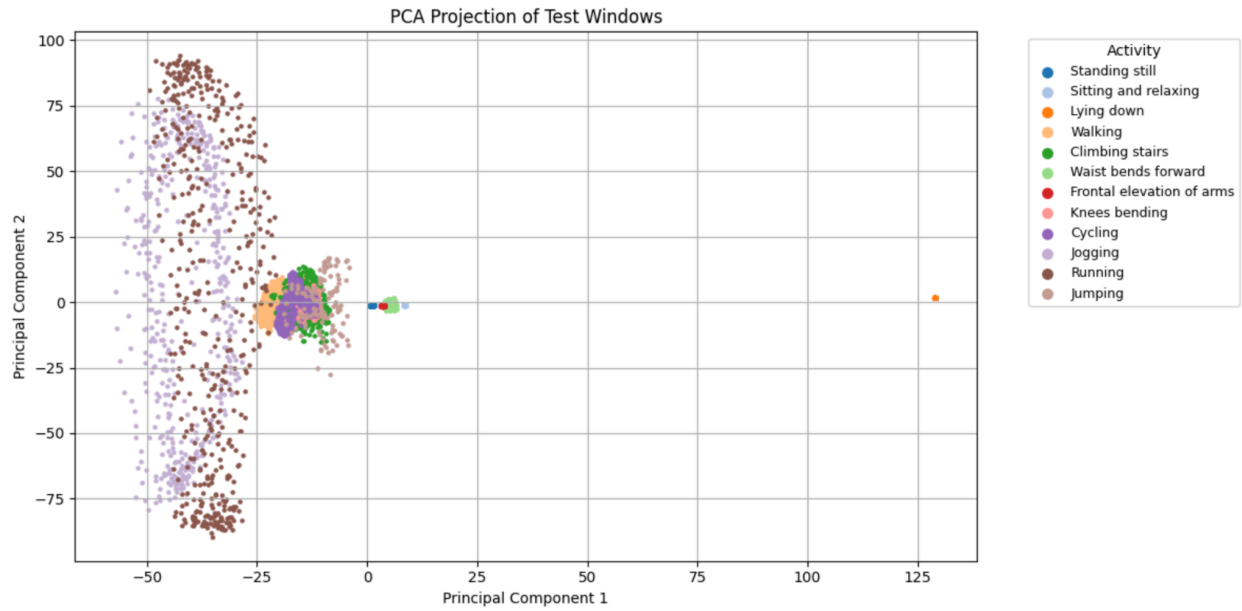
6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable. **The classifier results should only be trusted when the behavior is not an anomaly. If the anomaly is above the given threshold, the input and therefore the output should not be trusted.**

Section 2: Autoencoder Model Results

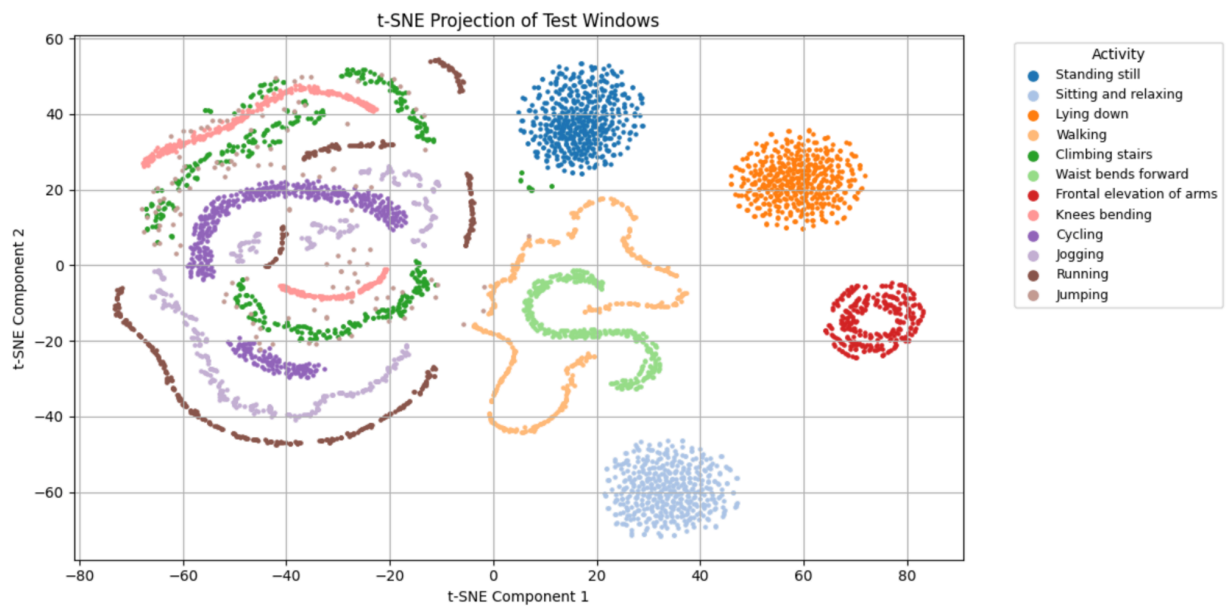
Training and validation loss curves:



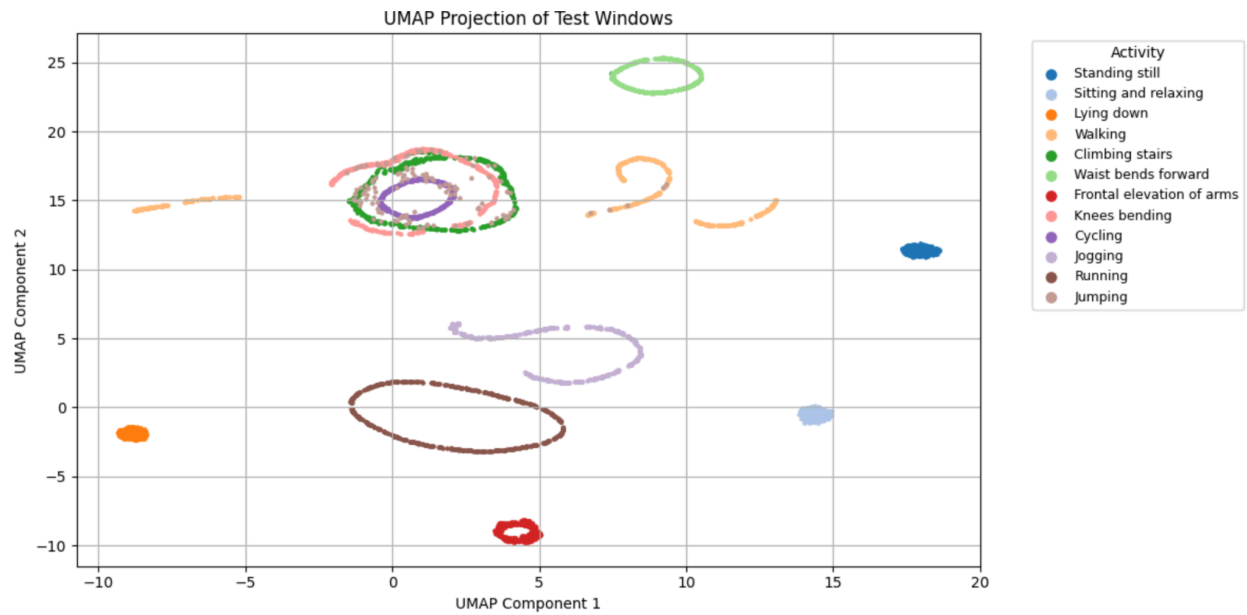
PCA Visualization:



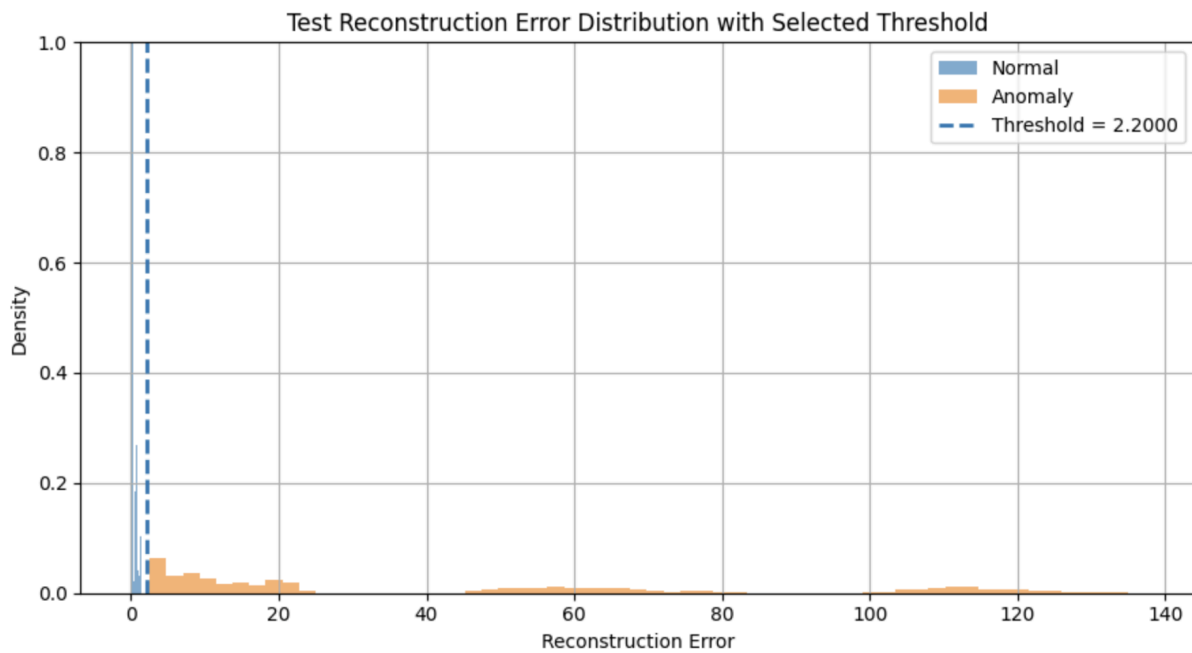
t-SNE Visualization:



UMAP Visualization:



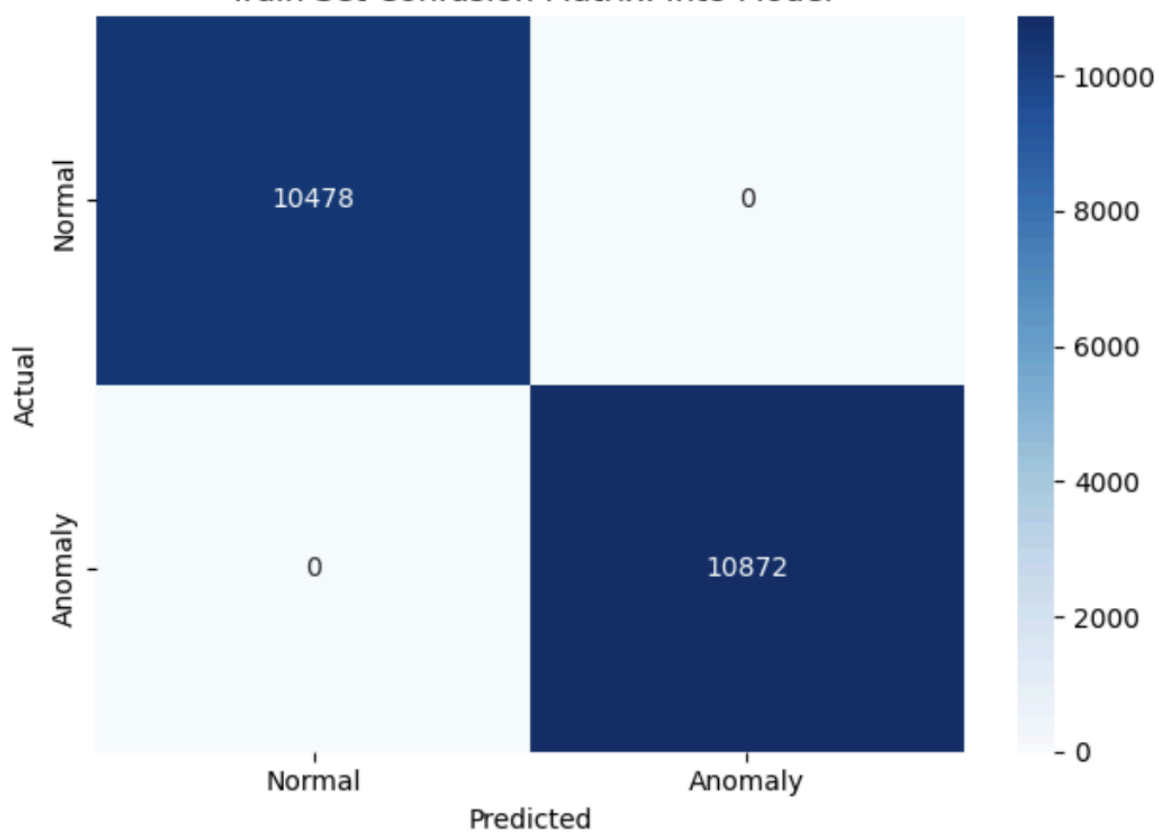
Reconstruction Error Distribution:



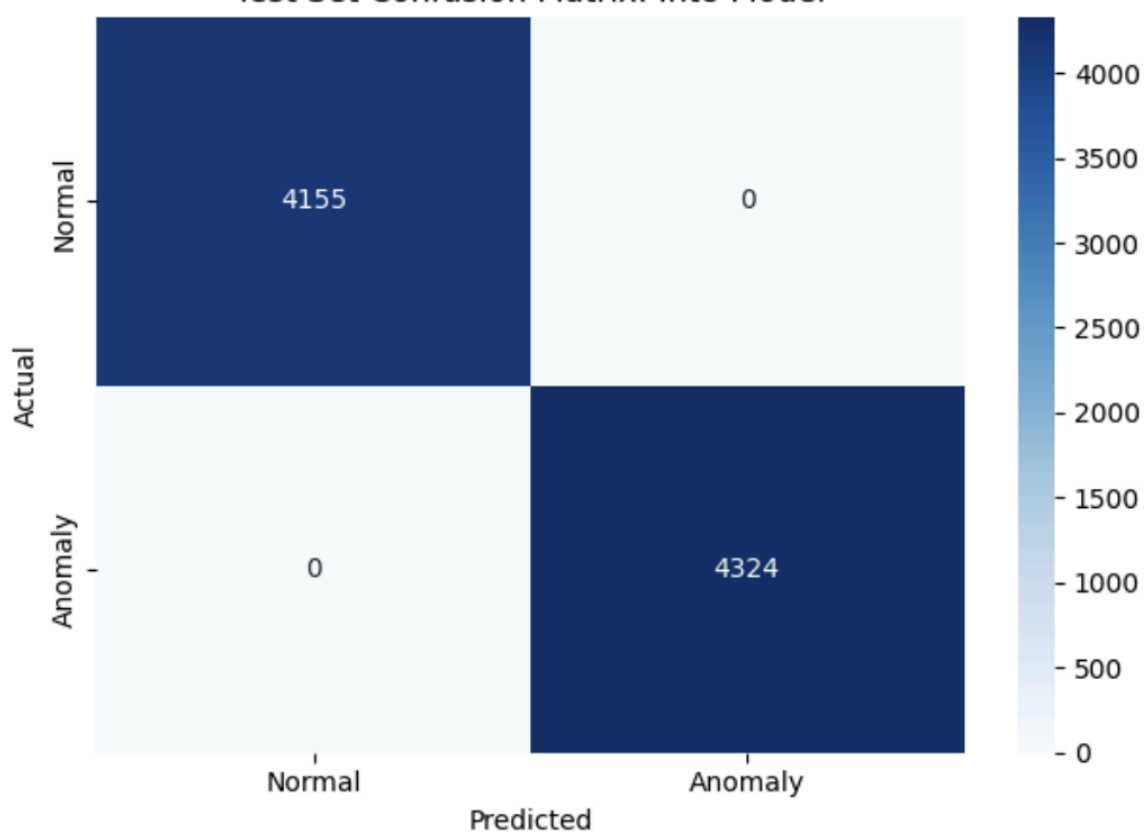
Selected Anomaly Threshold: 2.2

Confusion Matrix:

Train Set Confusion Matrix: Int8 Model



Test Set Confusion Matrix: Int8 Model



Classification Report:

Train set evaluation: int8 model				
	precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	10478
Anomaly (1)	1.00	1.00	1.00	10872
accuracy			1.00	21350
macro avg	1.00	1.00	1.00	21350
weighted avg	1.00	1.00	1.00	21350
Test set evaluation: int8 model				
	precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	4155
Anomaly (1)	1.00	1.00	1.00	4324
accuracy			1.00	8479
macro avg	1.00	1.00	1.00	8479
weighted avg	1.00	1.00	1.00	8479

Arduino Output:

```
Reconstruction error: 2.983502
Result: Anomaly detected
Reconstruction error: 2.975501
Result: Anomaly detected
Reconstruction error: 2.981633
Result: Anomaly detected
Reconstruction error: 2.973484
Result: Anomaly detected
Reconstruction error: 3.019222
```

```
Result: Normal activity
Reconstruction error: 0.124720
Result: Normal activity
Reconstruction error: 0.124719
Result: Normal activity
Reconstruction error: 0.124721
```

Section 3: Discussion Questions

What threshold did you choose for anomaly detection, and why? **I chose 2.2, because this threshold cleanly separated the normal and anomaly in the histogram.**

How does reconstruction error help distinguish normal and anomalous activity? **Since normal activity tends to have low reconstruction error, and anomalous activity will have unusually high reconstruction error, you can determine if activity is normal or not by looking at the reconstruction error.**

What information from the Python notebook must be preserved when deploying the model to Arduino? **The generated int8 quantized TFLite model, the threshold used, the data about what sensor data is used, and the window size.**

Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook? **It's important because they need to agree on feature alignment, and also they need the same assumptions for the 2.2 threshold to be accurate.**

What are the main limitations of this anomaly detection method when deployed on a microcontroller? **The limited RAM limits the amount you can store in buffers during processing, and the slower processing increases the latency from activity to detection.**